

Module 2 — Analyse et Conception

Architecture & Services Web

Semaine 9 · Cours 1 · GL-EN3-2026

Lundi 11 Mai 2026

Plan du cours

Semaine 9 · Cours 1

1 Architecture 3-tiers vs MVC

Deux niveaux d'abstraction complémentaires — déploiement vs organisation du code

2 Monolithe vs Microservices

Grille de décision et recommandation pour le contexte FDS-UEH

3 5 styles d'API

REST · GraphQL · gRPC · WebSockets · SSE — quand utiliser quoi

4 Contract-First & OpenAPI

Définir l'interface avant de coder — alignement frontend / backend

Projet 1 : ce cours alimente directement les sections 10 Architecture et 11 Choix technologiques

1

Architecture Logicielle

Principes · Patterns · Décisions

Architecture 3-tiers vs Pattern MVC

Deux niveaux d'abstraction différents — ils coexistent dans la même application

Dimension	Architecture 3-tiers	Pattern MVC
Niveau	Déploiement physique	Organisation du code
Couches	Présentation · Logique · Données	Model · View · Controller
Portée	Système entier	Application (souvent la couche UI)
Question	Où s'exécute chaque partie ?	Comment organiser le code UI ?
Exemple	React → Node API → Postgres	Express router → Controller → Model

En pratique : une application web utilise les deux. MVC s'applique à l'intérieur de la couche logique du modèle 3-tiers.

Le spectre des architectures

Du plus simple au plus distribué



Complexité croissante ->

Complexité opérationnelle

Faible

Faible

Élevée

Gérée par cloud

Scalabilité

Uniforme

Uniforme

Indépendante

Auto (0→N)

Déploiement

1 artefact

1 artefact

N services

Fonction

Taille équipe recommandée

1–5 pers.

3–10 pers.

10+ pers.

Variable

Monolithe — L'architecture tout-en-un

Définition · Cas d'usage · Avantages · Défis

Application déployée comme une seule unité. Toutes les fonctionnalités (UI, logique métier, accès aux données) s'exécutent dans un seul processus et sont déployées ensemble.

Cas d'usage

- MVP et prototypes
- Équipes < 5 personnes
- Domaine métier peu connu
- Budget DevOps limité
- Applications CRUD simples

Avantages

- Développement simple et rapide
- Un seul déploiement (un artefact)
- Transactions ACID faciles
- Faible latence (appels locaux)
- Debugging et tests simples

Défis

- Scalabilité uniforme seulement
- Couplage fort progressif
- Déploiements risqués (tout ou rien)
- Une seule stack technologique
- Équipes qui se bloquent

Monolithe Modulaire — Structure sans distribution

Définition · Cas d'usage · Avantages · Défis

Monolithe dont le code est organisé en modules internes aux frontières strictement respectées. Chaque module expose des interfaces définies ; aucun couplage direct entre modules n'est autorisé.

Cas d'usage	Avantages	Défis
■ Équipes 3–10 personnes ■ Domaine métier bien compris ■ Maintenabilité prioritaire ■ Transition microservices envisagée ■ Organisations avec une seule équipe	■ Déploiement simple (1 artefact) ■ Modules découplés → maintenable ■ Base de données partagée (ACID) ■ Migration progressive possible ■ Meilleur rapport simplicité/structure	■ Discipline d'équipe requise ■ Frontières peuvent être violées ■ Pas de scalabilité par module ■ Outils de vérification nécessaires ■ Partage de la base de données

Microservices — Services indépendants et distribués

Définition · Cas d'usage · Avantages · Défis

Application décomposée en services indépendants, chacun responsable d'un domaine métier, avec sa propre base de données, déployé et mis à l'échelle indépendamment des autres.

Cas d'usage	Avantages	Défis
■ Grandes équipes (1 équipe / service) ■ Domaine métier bien établi ■ Scalabilité différenciée critique ■ Haute disponibilité requise ■ Organisations type Conway's Law	■ Scalabilité indépendante par service ■ Déploiements sans coordination ■ Isolation des pannes ■ Autonomie technologique ■ Équipes vraiment indépendantes	■ Complexité opérationnelle élevée ■ Latence réseau inter-services ■ Cohérence des données difficile ■ Observabilité distribuée (tracing) ■ Coût d'infrastructure élevé

Serverless / FaaS — Fonctions éphémères dans le cloud

Définition · Cas d'usage · Avantages · Défis

Les fonctions (unité de déploiement minimale) s'exécutent sur une infrastructure entièrement gérée par le cloud provider. Facturation à l'invocation, scalabilité automatique de 0 à N, aucun serveur à administrer.

Cas d'usage	Avantages	Défis
■ Workloads event-driven (webhooks) ■ Tâches périodiques ou intermittentes ■ APIs à trafic variable / imprévisible ■ Pipelines de traitement de données ■ Startup avec budget infra limité	■ Aucune gestion de serveur ■ Scalabilité auto (0 à N instances) ■ Facturation à l'usage réel ■ Déploiement ultra-rapide ■ Time-to-market très court	■ Cold starts (latence initiale) ■ Vendor lock-in fort (AWS/GCP...) ■ Durée d'exécution limitée ■ Debugging complexe (stateless) ■ Coût élevé si charge continue

Tableau comparatif

Vue synthétique pour guider le choix

	Monolithe	Monolithe Modulaire	Microservices	Serverless
Déploiement	1 artefact	1 artefact	N services	Fonctions
Scalabilité	Uniforme	Uniforme	Indépendante	Auto (0→N)
Complexité ops	Faible	Faible	Élevée	Gérée cloud
Cohérence données	ACID	ACID	Eventuelle	Externe requis
Taille équipe	1–5	3–10	10+	Variable
Cas typique	MVP, startup	Domaine établi	Grande org.	Event-driven
Dette technique	Risque élevé	Faible	Faible	Faible

Autres architectures à connaître

Pour approfondir — pistes de recherche

Event-Driven Architecture (EDA)

Les composants communiquent via des événements publiés sur un bus de messages (Kafka, RabbitMQ). Le producteur ignore qui consomme.

Exemple

Systèmes de notifications, pipelines analytiques, e-commerce (panier → paiement → livraison)

SOA — Service-Oriented Architecture

Services partagés communiquant via un Enterprise Service Bus (ESB). Prédécesseur des microservices, encore présent dans les grands systèmes d'entreprise.

Exemple

Systèmes bancaires, ERP d'entreprise (SAP), administrations publiques

Architecture Hexagonale (Ports & Adapters)

Le cœur métier (domaine) est isolé de l'infrastructure (DB, UI, APIs externes) via des ports et adaptateurs. Permet de tester le métier indépendamment.

Exemple

Applications avec logique métier complexe, DDD (Domain-Driven Design)

CQRS + Event Sourcing

CQRS sépare les opérations de lecture (Query) et d'écriture (Command). Event Sourcing stocke l'état comme une séquence d'événements immuables.

Exemple

Systèmes financiers (audit complet), applications collaboratives, historique complet requis

Comment choisir ?

Posez-vous ces questions :

Votre équipe fait-elle < 5 personnes ou travaillez-vous sur un MVP ?

->

Monolithe

Avez-vous un domaine bien compris et besoin de maintenabilité élevée ?

->

Monolithe Modulaire

Avez-vous 10+ développeurs et une scalabilité différenciée critique ?

->

Microservices

Vos traitements sont-ils déclenchés par des événements / périodiques ?

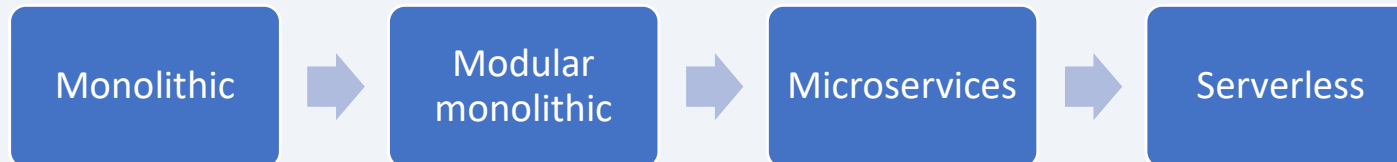
->

Serverless

Règle d'or : commencez simple (monolithe), migrez quand la douleur est réelle — pas avant.

Monolithe vs Microservices (Shortcut)

Grille de décision



Critère	Monolithe recommandé	Microservices recommandé
Taille équipe	< 5 développeurs	Plusieurs équipes autonomes
Maturité domaine	Domaine mal compris (MVP)	Frontières métier bien définies
Scalabilité	Besoins uniformes	Besoins différenciés par service
Complexité ops	Faible tolérance opérationnelle	Équipe DevOps disponible
Phase projet	MVP · Prototype	Produit en production établi

Recommandation FDS-UEH : Monolithe modulaire pour Projet 1 — modules bien séparés dans une base de code unique.

2

Services Web & API

REST · GraphQL · gRPC · WebSockets · SSE

REST — Representational State Transfer

Définition · Cas d'usage · Avantages · Défis

Style architectural utilisant HTTP avec des ressources identifiées par des URLs. Interface uniforme (GET / POST / PUT / DELETE), communication sans état, représentation des données en JSON ou XML.

Cas d'usage

- APIs publiques et intégrations B2B
- Applications CRUD classiques
- Mobile backends (iOS, Android)
- Portails avec documentation OpenAPI
- Systèmes avec besoins de cache HTTP

Avantages

- Standard universel, écosystème mature
- Cache HTTP natif (GET cachable)
- Stateless = scalabilité horizontale
- Documentation facile (Swagger/OpenAPI)
- Compatible avec tous les clients

Défis

- Over-fetching / under-fetching
- Versioning d'API complexe (/v1, /v2)
- Pas adapté au temps réel
- Multiples requêtes pour données liées
- Pas de typage fort natif

GraphQL — Langage de requête flexible

Définition · Cas d'usage · Avantages · Défis

Langage de requête permettant au client de spécifier exactement les données dont il a besoin. Un seul endpoint, le client contrôle la forme et la profondeur de la réponse. Développé par Meta (2015).

Cas d'usage	Avantages	Défis
■ Applications mobiles (données variées) ■ Dashboards avec vues complexes ■ Pattern BFF (Backend For Frontend) ■ Agrégation de plusieurs microservices ■ Équipes frontend/backend indépendantes	■ Pas d'over-fetching ni under-fetching ■ Auto-documenté (introspection) ■ Une seule version d'API ■ Flexible : le client contrôle tout ■ Idéal pour interfaces riches	■ Cache HTTP difficile à mettre en place ■ Problème N+1 (requêtes en boucle) ■ Courbe d'apprentissage côté serveur ■ Overkill pour APIs simples ■ Erreurs HTTP non standard (200 partout)

gRPC — Remote Procedure Call haute performance

Définition · Cas d'usage · Avantages · Défis

Framework RPC de Google utilisant HTTP/2 et Protocol Buffers (protobuf). Les contrats sont définis dans des fichiers .proto. Communication binaire compressée, streaming natif bidirectionnel.

Cas d'usage	Avantages	Défis
■ Communication inter-services (microservices) ■ Systèmes haute performance et faible latence ■ Streaming bidirectionnel (audio, vidéo) ■ IoT et systèmes embarqués ■ Services internes non exposés au navigateur	■ Performance maximale (binaire + HTTP/2) ■ Streaming bidirectionnel natif ■ Contrat typé fort (.proto = source de vérité) ■ Code client auto-généré (tous langages) ■ Multiplexage des requêtes (HTTP/2)	■ Non natif dans les navigateurs ■ Débogage difficile (binaire non lisible) ■ Courbe d'apprentissage (protobuf, .proto) ■ Moins adapté aux APIs publiques ■ Outillage moins mature que REST

WebSockets — Communication bidirectionnelle temps réel

Définition · Cas d'usage · Avantages · Défis

Protocole de communication full-duplex persistant. Après un handshake HTTP initial (Upgrade), une connexion TCP ouverte permet au serveur ET au client d'envoyer des messages à tout moment, sans nouvelle requête.

Cas d'usage	Avantages	Défis
■ Chat et messagerie instantanée ■ Jeux multijoueurs en ligne ■ Collaboration temps réel (type Google Docs) ■ Trading et cours boursiers live ■ Notifications push bidirectionnelles	■ Temps réel vrai (latence minimale) ■ Bidirectionnel = client et serveur actifs ■ Connexion persistante (pas de re-handshake) ■ Adapté aux échanges à haute fréquence ■ Événements serveur vers client instantanés	■ Connexions persistantes = ressources serveur ■ Scalabilité complexe (état par connexion) ■ Gestion déconnexion / reconnexion requise ■ Pas cachable, pas compatible proxies HTTP ■ Sécurité : protéger le canal ouvert (WSS)

SSE — Server-Sent Events (flux unidirectionnel)

Définition · Cas d'usage · Avantages · Défis

Mécanisme HTTP standard permettant au serveur d'envoyer un flux continu d'événements vers le client via une connexion HTTP longue.
Communication unidirectionnelle : serveur → client uniquement.

Cas d'usage	Avantages	Défis
■ Tableaux de bord et métriques live ■ Flux d'actualités et alertes système ■ Progression de tâches longues (upload, build) ■ Mises à jour de statut en temps réel ■ IA generative (streaming de tokens LLM)	■ Simple : HTTP natif, aucune bibliothèque ■ Reconnexion automatique intégrée ■ Compatible avec les proxies HTTP ■ ID d'événements pour reprendre le flux ■ Faible consommation vs WebSockets	■ Unidirectionnel seulement (serveur → client) ■ Limite connexions simultanées (HTTP/1.1) ■ Pas de binaire natif (texte uniquement) ■ Moins adapté aux échanges fréquents ■ Moins connu et moins bien supporté côté server frameworks

Stateless vs Stateful

Deux modèles de gestion de l'état — un choix architectural fondamental

Stateless (sans état)

Le serveur ne garde aucune information entre deux requêtes. Chaque requête est autonome et contient tout ce qu'il faut pour être traitée (token, contexte...).

Exemples : REST · gRPC · Serverless / FaaS

- + Scalabilité horizontale facile
- + Cache HTTP possible
- + Reprise simple après panne
- + Load balancing sans affinité
- Requêtes plus volumineuses (token à chaque fois)
- Pas adapté aux interactions continues

Stateful (avec état)

Le serveur maintient un état de la session entre les échanges. Les interactions successives dépendent du contexte accumulé — connexion persistante ou session côté serveur.

Exemples : WebSockets · Sessions HTTP · Jeux en ligne

- + Interactions continues sans répétition du contexte
- + Performances pour échanges à haute fréquence
- + Idéal pour protocoles bidirectionnels
- Scalabilité complexe (session sticky)
- Panne serveur = perte d'état possible
- Load balancing nécessite affinité de session

Règle : REST (stateless) par défaut — WebSockets (stateful) uniquement si le temps réel bidirectionnel est une exigence fonctionnelle réelle.

Les 5 styles d'API

Choisir le bon outil selon le cas d'usage

Style	Protocole	Cas d'usage typique	Format
REST	HTTP/HTTPS	CRUD, APIs publiques, intégrations B2B	JSON / XML
GraphQL	HTTP	Clients mobiles, données variées, réduire l'over-fetching	JSON
gRPC	HTTP/2	Communication inter-services, haute performance	Protocol Buffers
WebSockets	TCP (HTTP upg.)	Chat, collaboration temps réel, jeux multijoueurs	JSON / binaire
SSE	HTTP	Notifications push, dashboards live (serveur vers client)	text/event-stream

Ne pas oublier : Stateless et Stateful

Tableau Comparatif

Types d'API

REST

GraphQL

gRPC

WebSockets

SSE

Module 2 — Analyse & Conception | Semaine 9

Critères techniques · Cas d'usage · Guide de choix

Comparatif Technique

Critère	REST	GraphQL	gRPC	WebSockets	SSE
Protocole	HTTP/1.1 & 2	HTTP/1.1 & 2	HTTP/2 requis	WS (TCP)	HTTP (SSE)
Format données	JSON / XML	JSON	Protobuf (bin.)	Texte / Binaire	Texte (UTF-8)
Stateless / Stateful	Stateless	Stateless	Stateless	Stateful	Semi-stateful
Cache HTTP natif	Oui (GET)	Limité (GET)	Non	Non	Oui (GET)
Streaming	Non natif	Subscriptions	Bidirectionnel	Full-duplex	Serveur → client
Support navigateur	Natif (fetch)	Natif + lib	grpc-web requis	Natif (WS API)	Natif (EventSource)
Complexité impl.	Faible	Moyenne	Elevée (schema)	Moyenne	Faible

Comparatif Usage & Cas Pratiques

Critère	REST	GraphQL	gRPC	WebSockets	SSE
Temps réel	Non adapté	Subscriptions (WS)	Streaming natif	Idéal (full-duplex)	Bon (unidirectionnel)
APIs publiques	Idéal	Très bon	Rarement	Peu courant	Usage limité
Perfor-mances	Bonnes	Moyennes	Excellentes	Excellentes	Bonnes
Courbe d'apprentissage	Douce	Moderate	Élevée	Moderate	Douce
Cas typique	CRUD, API mobiles	BFF, UI flexibles	Microservices internes	Chat, jeux, collab.	Notifs, dashboards
Versioning	Facile (URI v1/v2)	Évolutif (schema)	Proto backward compat.	Difficile	Difficile
Documentation	OpenAPI / Swagger	Introspection schema	Proto files	Manuelle	Manuelle

Guide de Choix — Quel type d'API pour quel besoin ?

REST	GraphQL	gRPC	WebSockets	SSE
API CRUD publique / mobile	UI flexible, many endpoints	Microservices internes haute perf.	Communication bidirectionnelle temps réel	Notifications serveur → client
Simple, cacheable, bien outillé. Standard universel pour les APIs exposées.	Le client demande exactement ce dont il a besoin. Idéal pour les BFF.	Protobuf + HTTP/2 = latence minimale. Typage fort entre services.	Full-duplex persistant. Chat, collaboration live, jeux en ligne.	Léger, natif HTTP. Dashboards live, alertes, flux d'événements.

Règle d'or : Commencez par REST. Adoptez GraphQL si les clients ont des besoins variés. Passez à gRPC pour les services internes critiques. WebSockets & SSE pour le temps réel.

REST — L'interface uniforme

Les 4 opérations HTTP fondamentales

Méthode	Action	Idempotent	Exemple URL
GET	Lire une ressource	Oui	GET /users/42
POST	Créer une ressource	Non	POST /users
PUT	Remplacer une ressource	Oui	PUT /users/42
DELETE	Supprimer une ressource	Oui	DELETE /users/42

Ressource = nom (ex: /users, /products) · Méthode HTTP = verbe · Stateless = pas de session côté serveur

Contract-First & OpenAPI

Définir l'interface avant de coder

Approche Contract-First

1 Définir le contrat API

Endpoints, paramètres, schémas de données

2 Aligner frontend + backend

Référence partagée — pas d'ambiguïté

3 Mocker le backend

Le frontend travaille sans attendre l'implémentation

4 Générer & tester

SDK client, stubs serveur, tests du contrat automatisés

OpenAPI Specification (OAS)

```
openapi: "3.1.0"
info:
  title: FDS API
  version: "1.0"
paths:
  /users:
    get:
      summary: Liste des utilisateurs
      responses:
        '200': OK
```

Swagger UI génère automatiquement
une documentation interactive
à partir du fichier openapi.yaml.

spec.openapis.org

3

Vers le Projet 1

Section 10 — Architecture technique

Section 10 — Architecture technique

Le diagramme de composants UML : livrables attendus

Ce que le diagramme représente

- 1 Les composants du système (frontend, backend, base de données, services externes)
- 2 Les interfaces exposées par chaque composant
- 3 Les dépendances et les flux de communication
- 4 Les choix de déploiement (ex : client séparé du serveur API)

Contenu attendu dans la section 10

Diagramme de composants

UML ou schéma équivalent — clair et lisible

Pattern architectural

3-tiers + MVC ? Monolithe modulaire ? Justifié.

Style d'API choisi

REST / GraphQL / autre — avec rationale

Technologies par couche

ex : React (UI) · Express (API) · Postgres (BDD)

Schéma de communication

Qui appelle qui ? Synchrone ou asynchrone ?

Synthèse du Cours 1

3T

3-tiers = déploiement / MVC = organisation du code — les deux coexistent

M/M

Monolithe modulaire recommandé pour Projet 1 — simple, maintenable, adapté à vos équipes

API

REST est le choix par défaut — stateless, HTTP, JSON, interface uniforme

CF

Contract-First : définir openapi.yaml avant de coder — aligne frontend et backend

§10

Diagramme de composants + justification du pattern = livrable §§10 du Projet 1

Cours 2 — Jeudi 15 Mai (On fini le Contenu du cours 1 d'abord)

Sécurité par conception (OWASP Top 10) · AuthN vs AuthO (JWT, OAuth2, RBAC) · WCAG 2.1

Performance & Scalabilité · Revue ATAM simplifiée · Section §§11 Choix technologiques